```
separate (IDL.SEM_PHASE)¶--|==========================================================================================|¶--|¬¬¬¬¬¬¬¬¬¬|¶--|¬¬¬¬D
EF_UTIL¬¬¬¬¬¬|¶--|¬¬¬¬¬¬¬¬¬¬|¶--|==========================================================================================|¶package body DEF_U
TIL is¶¶  use VIS_UTIL; -- FOR DEBUG (NODE_REP)¶  use EXPRESO; -- FOR GET_NAME_DEFN¶¶--|==============================================================
====================================¶¶--|¬¬¬¬HEADER_IS_HOMOGRAPH¶¶--|_________________________________________________________________________________
_____________¶  function HEADER_IS_HOMOGRAPH ( HEADER_1 :TREE; PARAM_S_2 :TREE; RESULT_TYPE_2¬: TREE :=¬TREE_VOID¬) return BOOLEAN is¶    KIND_1¬: c
onstant NODE_NAME¬:= HEADER_1.TY;¶  begin¶¶    if KIND_1 not in CLASS_SUBP_ENTRY_HEADER or else PARAM_S_2 = TREE_VOID then¬¬¬--| IF HEADER_1 IS NON_OV
ERLOADABLE OR PARAM_S_2 IS VOID¶      return True;¬¬¬¬¬¬¬¬¬¬--| ILS SONT HOMOGRAPHES¶    end if;¶¶    if (KIND_1 = DN_FUNCTION_SPEC) xor (RESULT_TYPE_2
 /= TREE_VOID) then¬¬¬--| L'UN FONCTION L'AUTRE NON¶      return FALSE;¬¬¬¬¬¬¬¬¬¬--| ILS NE SONT PAS¬HOMOGRAPHES¶    end if;¶¶    if KIND_1 = DN_FUNCTI
ON_SPEC then¬¬¬¬¬¬¬¬--| DEUX FONCTIONS¶      if GET_BASE_TYPE( D( AS_NAME, HEADER_1 ) ) /= GET_BASE_TYPE( RESULT_TYPE_2 ) then¬¬¬--| TYPES¬RETOURNES¬DIF
FERENTS¶        return FALSE;¬¬¬¬¬¬¬¬¬--| ILS NE SONT PAS¬HOMOGRAPHES¶      end¬if;¶    end if;¶¶    return IS_SAME_PARAMETER_PROFILE( D( AS_PARAM_S, H
EADER_1 ), PARAM_S_2 );¬¬¬¬--| COMPARER LES PROFILS DE PARAMTRES¶  end HEADER_IS_HOMOGRAPH;¶--|#############################################################
##########################################¶¶--|¬¬¬¬MAKE_DEF_FOR_ID¶¶--|_______________________________________________________________________________
________________¶  function MAKE_DEF_FOR_ID ( ID : TREE;¬H : H_TYPE ) return¬TREE is¶    SYMREP¬: constant TREE¬:= D( LX_SYMREP, ID¬);¶    DEF¬¬: TRE
E¬¬:= MAKE( DN_DEF );¶  begin¶    if H.REGION_DEF¬/= TREE_VOID and then ID.TY in CLASS_SOURCE_NAME then¶      D( XD_REGION,¬ID, D( XD_SOURCE_NAME, H.R
EGION_DEF ) );¶    end if;¶¶    D(  XD_HEADER,¬    DEF,¬TREE_TRUE¬   );¶    D(  XD_SOURCE_NAME, DEF,¬ID¬   );¶    D(  XD_REGION_DEF,  DEF,¬H.REGION_DE
F );¶    DB( XD_IS_IN_SPEC,  DEF,¬H.IS_IN_SPEC );¶    DB( XD_IS_USED,¬    DEF,¬FALSE¬   );¶    DI( XD_LEX_LEVEL,   DEF,¬0¬   );¶¶    LIST (SYMREP, INS
ERT (LIST (SYMREP), DEF));¶    return DEF;¶  end MAKE_DEF_FOR_ID;¶--|##################################################################################
################¶¶--|¬¬¬¬CHECK_UNIQUE_SOURCE_NAME_S¶¶--|__________________________________________________________________________________________________
___¶  procedure CHECK_UNIQUE_SOURCE_NAME_S ( SOURCE_NAME_S :TREE ) is¶    SOURCE_NAME_LIST¬: SEQ_TYPE¬:= LIST( SOURCE_NAME_S );¶    SOURCE_NAME¬¬: TRE
E;¶  begin¶    while¬not IS_EMPTY( SOURCE_NAME_LIST ) loop¶      POP( SOURCE_NAME_LIST, SOURCE_NAME );¶      CHECK_UNIQUE_DEF( GET_DEF_FOR_ID(¬SOURCE_
NAME ) );¶    end loop;¶  end CHECK_UNIQUE_SOURCE_NAME_S;¶--|####################################################################################
########¶¶--|¬¬¬¬CHECK_CONSTANT_ID_S¶¶--|______________________________________________________________________________________________¶  procedure
 CHECK_CONSTANT_ID_S (SOURCE_NAME_S :TREE; H :H_TYPE ) is¶    SOURCE_NAME_LIST¬: SEQ_TYPE¬:= LIST( SOURCE_NAME_S );¶    SOURCE_NAME¬¬: TREE;¶  begin¶
   while¬not IS_EMPTY( SOURCE_NAME_LIST ) loop¶      POP( SOURCE_NAME_LIST, SOURCE_NAME );¶      CHECK_CONSTANT_DEF( GET_DEF_FOR_ID( SOURCE_NAME ), H
);¶    end loop;¶  end CHECK_CONSTANT_ID_S;¶--|#################################################################################################¶¶--|¬
¬¬¬GET_DEF_FOR_ID¶¶--|__________________________________________________________________________________________________¶  function GET_DEF_FOR_ID ( ID
 :TREE ) return TREE is¶    DEFLIST¬: SEQ_TYPE¬:= LIST( D( LX_SYMREP, ID ) );¶    DEF¬¬: TREE;¶  begin¶    while¬not IS_EMPTY( DEFLIST ) loop¶      PO
P( DEFLIST,¬DEF );¶¶      if D (XD_SOURCE_NAME, DEF) = ID then¶        return DEF;¶      end¬if;¶    end loop;¶¶    Put_Line ("!! NO DEF FOR ID - " &
PRINT_NAME (D (LX_SYMREP, ID)));¶    raise¬Program_Error;¶  end GET_DEF_FOR_ID;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||¶      --|¶  function GET_PRIOR_DEF (DEF¬: TREE) return TREE¬is¶    REGION_DEF : constant TREE := D (XD_REGION_DEF, DEF)
;¶    HEADER     : constant TREE := D (XD_HEADER, DEF);¶    DEFLIST    : SEQ_TYPE¬ := LIST (D (LX_SYMREP, D (XD_SOURCE_NAME, DEF)));¶    PRIOR_DEF  :
TREE;¶  begin¶    while¬not IS_EMPTY (DEFLIST) loop¶      POP¬(DEFLIST,¬PRIOR_DEF);¶      if PRIOR_DEF /= DEF and¬then D (XD_REGION_DEF, PRIOR_DEF) =
REGION_DEF then¶        return PRIOR_DEF;¶      end¬if;¶    end loop;¶¶    return TREE_VOID;¶  end GET_PRIOR_DEF;¶      --|||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function GET_PRIOR_HOMOGRAPH_DEF (DEF¬: TREE) return TREE¬is¶    HEADER :
 TREE := D (XD_HEADER, DEF);¶  begin¶    if HEADER.TY = DN_FUNCTION_SPEC then¶      return GET_PRIOR_HOMOGRAPH_DEF (DEF, D (AS_PARAM_S, HEADER), D (AS
_NAME, HEADER));¶    else -- SINCE IT IS A PROCEDURE_SPEC OR AN ENTRY¶      return GET_PRIOR_HOMOGRAPH_DEF (DEF, D (AS_PARAM_S, HEADER));¶    end if;¶
  end GET_PRIOR_HOMOGRAPH_DEF;¶¶  function GET_PRIOR_HOMOGRAPH_DEF (DEF, PARAM_S : TREE; RESULT_TYPE : TREE := TREE_VOID)¬return TREE is¶¬      -- NOT
E: DOES¬NOT FIND DERIVED AND BUILTIN SUBPROGRAMS¶    REGION_DEF : constant TREE := D (XD_REGION_DEF, DEF);¶    DEFLIST    : SEQ_TYPE¬ := LIST (D (LX_S
YMREP, D (XD_SOURCE_NAME, DEF)));¶    PRIOR_DEF  : TREE;¶  begin¶    while¬not IS_EMPTY (DEFLIST) loop¶      POP¬(DEFLIST,¬PRIOR_DEF);¶      if PRIOR_
DEF /= DEF and¬then D (XD_SOURCE_NAME, PRIOR_DEF).TY /= DN_BLTN_OPERATOR_ID and then¬D (XD_SOURCE_NAME, PRIOR_DEF).TY not in¬CLASS_ENUM_LITERAL and th
en D¬(XD_REGION_DEF, PRIOR_DEF) = REGION_DEF¶        and then HEADER_IS_HOMOGRAPH (D (XD_HEADER, PRIOR_DEF), PARAM_S, RESULT_TYPE) and then (D (XD_SOUR
CE_NAME, PRIOR_DEF).TY not in CLASS_SUBPROG_NAME or else D (SM_UNIT_DESC, D (XD_SOURCE_NAME, PRIOR_DEF)).TY /= DN_DERIVED_SUBPROG)¶      then¶
return PRIOR_DEF;¶      end¬if;¶    end loop;¶¶    return TREE_VOID;¶  end GET_PRIOR_HOMOGRAPH_DEF;¶      --||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function GET_DEF_IN_REGION (ID : TREE; H : H_TYPE) return¬TREE is¶    REGION_DEF : cons
tant TREE := H.REGION_DEF;¶    DEFLIST    : SEQ_TYPE¬ := LIST (D (LX_SYMREP, ID));¶    PRIOR_DEF  : TREE;¶  begin¶    while¬not IS_EMPTY (DEFLIST) loo
p¶      POP¬(DEFLIST,¬PRIOR_DEF);¶      if D (XD_REGION_DEF, PRIOR_DEF) =¬REGION_DEF then¶        return PRIOR_DEF;¶      end¬if;¶    end loop;¶¶    r
eturn TREE_VOID;¶  end GET_DEF_IN_REGION;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      -
-|¶  procedure CHECK_UNIQUE_DEF (SOURCE_DEF : TREE) is¶    PRIOR_DEF    : constant TREE := GET_PRIOR_DEF (SOURCE_DEF);¶    SOURCE_NAME : TREE;¶  begin¶
    if PRIOR_DEF /=¬TREE_VOID¬then¶      SOURCE_NAME := D (XD_SOURCE_NAME,¬SOURCE_DEF);¶      ERROR (D (LX_SRCPOS, SOURCE_NAME), "DEFINITION IS NOT UN
IQUE - " & PRINT_NAME (D (LX_SYMREP, SOURCE_NAME)));¶      D (XD_HEADER,¬SOURCE_DEF, TREE_FALSE);¶    else¶      D (XD_HEADER,¬SOURCE_DEF, TREE_VOID);

¶    end if;¶  end CHECK_UNIQUE_DEF;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶
procedure CHECK_CONSTANT_DEF (SOURCE_DEF : TREE; H : H_TYPE) is¶    SOURCE_ID : constant TREE¬:= D (XD_SOURCE_NAME, SOURCE_DEF);¶    PRIOR_DEF : TREE;
¶    PRIOR_ID  : TREE;¶  begin¶¬      -- IF WE ARE NOT IN PRIVATE PART OF A PACKAGE¶    if (H.IS_IN_SPEC) or (H.IS_IN_BODY)¬then¶¶¬¬      -- CHECK FOR U
NIQUENESS AND RETURN¶      CHECK_UNIQUE_DEF (SOURCE_DEF);¶      return;¶    end if;¶¶¬      -- GET PRIOR DEF, IF ANY¶    PRIOR_DEF := GET_PRIOR_DEF (S
OURCE_DEF);¶    if PRIOR_DEF = TREE_VOID then¶      MAKE_DEF_VISIBLE (SOURCE_DEF);¶      return;¶    else¶      PRIOR_ID := D¬(XD_SOURCE_NAME, PRIOR_D
EF);¶    end if;¶¶¬      -- IF PRIOR DEF IS NOT FOR A DEFERRED CONSTANT¶¬      -- WHICH DOES¬NOT YET HAVE A FULL¬DECLARATION¶    if PRIOR_ID.TY /= DN_
CONSTANT_ID or¬else D (SM_INIT_EXP, PRIOR_ID) /= TREE_VOID then¶¶¬¬      -- REPEAT UNIQUENESS CHECK TO PUT OUT ERROR MESSAGE AND¬RETURN¶      CHECK_UNIQ
UE_DEF (SOURCE_DEF);¶      return;¶    end if;¶¶¬      -- YES, IT IS¬A FULL DECLARATION OF A DEFERRED CONSTANT¶¶¬      -- CHECK CONFORMANCE OF¬DISCRIM
INANT LISTS¶¬      -- AND REMOVE¬DEF'S FOR¬DUPLICATED NAMES¶¬      -- $$$$$$ STUB -- MUST DO THIS CHECK --- $$$$$$$¶¬      --¬    IF KIND ( D ( SM_TYP
E_SPEC, SOURCE_ID)) = DN_RECORD THEN¶¬      --¬        CONFORM_PARAMETER_LISTS¶¬      --¬¬        ( D¬( SM_DISCRIMINANT_S, PRIOR_ID)¶¬      --¬¬        ,
D¬( SM_DISCRIMINANT_S, SOURCE_ID) );¶¬      --¬    ELSE¶¬      --¬        CONFORM_PARAMETER_LISTS¶¬      --¬¬        ( D¬( SM_DISCRIMINANT_S, PRIOR_ID)¶
¬      --¬¬        , CONST_VOID );¶¬      --¬    END IF;¶¶¬      -- MAKE SOURCE DEF VISIBLE AND RETURN¶    REMOVE_DEF_FROM_ENVIRONMENT (SOURCE_DEF);¶
 D (SM_FIRST, SOURCE_ID, PRIOR_ID);¶    return;¶  end CHECK_CONSTANT_DEF;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||¶      --|¶  procedure CHECK_TYPE_DEF (SOURCE_DEF : TREE; H : H_TYPE) is¶    PRIOR_DEF : constant TREE¬:= GET_PRIOR_DEF (SOURC
E_DEF);¶¶    SOURCE_ID : TREE;¶    PRIOR_ID  : TREE;¶¶  begin¶¬      -- IF THERE IS NO PRIOR¬DEF THEN¶    if PRIOR_DEF = TREE_VOID then¶¶¬¬      -- MAKE
 SOURCE DEF VISIBLE AND RETURN¶      MAKE_DEF_VISIBLE (SOURCE_DEF);¶      return;¶    end if;¶¶¬      -- GET ID'S CORRESPONDING TO DEF'S¶    SOURCE_ID
 := D (XD_SOURCE_NAME, SOURCE_DEF);¶    PRIOR_ID  := D (XD_SOURCE_NAME, PRIOR_DEF);¶¶¬      -- IF VALID FULL DECLARATION FOR PRIVATE TYPE¶    if PRIOR
_ID.TY in DN_PRIVATE_TYPE_ID .. DN_L_PRIVATE_TYPE_ID and then not H.IS_IN_SPEC and then¬not H.IS_IN_BODY then¶¶      declare¶        PRIVATE_NODE : co
nstant TREE :=¬D (SM_TYPE_SPEC, PRIOR_ID);¶      begin¶¬¬¬  -- IF NOT ALREADY¬DECLARED¶        if D (SM_TYPE_SPEC, PRIVATE_NODE) = TREE_VOID then¶¶¬¬¬
¬-- MAKE THIS THE FULL TYPE DECLARATION¶¬D (SM_TYPE_SPEC, PRIVATE_NODE, D (SM_TYPE_SPEC, SOURCE_ID));¶¬D (SM_FIRST, SOURCE_ID, PRIOR_ID);¶¶¬¬¬¬¬-- CHEC
K CONFORMANCE OF DISCRIMINANT LISTS¶¬¬¬¬¬-- AND REMOVE DEF'S¬FOR DUPLICATED NAMES¶¬if D (SM_TYPE_SPEC,¬SOURCE_ID).TY = DN_RECORD then¶¬  CONFORM_PARAME
TER_LISTS (D (SM_DISCRIMINANT_S, PRIOR_ID),¬D (SM_DISCRIMINANT_S, SOURCE_ID));¶¬else¶¬  CONFORM_PARAMETER_LISTS (D (SM_DISCRIMINANT_S, PRIOR_ID),¬TREE
_VOID);¶¬end if;¶¶¬¬¬¬¬-- MAKE SOURCE DEF VISIBLE AND RETURN¶¬MAKE_DEF_VISIBLE (SOURCE_DEF);¶¬return;¶        end if;¶      end;¶    end if;¶¶¬      --
 IF POSSIBLE VALID FULL DECLARATION FOR INCOMPLETE TYPE DECLARATION¶    if PRIOR_ID.TY = DN_TYPE_ID and then not H.IS_IN_SPEC and then not H.IS_IN_BOD
Y then¶¶      declare¶        INCOMPLETE_NODE : constant TREE¬:= D (SM_TYPE_SPEC,¬PRIOR_ID);¶      begin¶¬¬¬  -- IF PRIOR ID IS¬INCOMPLETE AND NOT ALR
EADY DECLARED¶        if INCOMPLETE_NODE.TY¬= DN_INCOMPLETE and¬then D (XD_FULL_TYPE_SPEC, INCOMPLETE_NODE) = TREE_VOID then¶¶¬¬¬¬¬-- MAKE THIS THE FUL
L TYPE DECLARATION¶¬D (XD_FULL_TYPE_SPEC, INCOMPLETE_NODE, D (SM_TYPE_SPEC, SOURCE_ID));¶¬D (SM_FIRST, SOURCE_ID, PRIOR_ID);¶¶¬¬¬¬¬-- REMOVE¬SOURCE DEF
 FROM ENVIRONMENT AND RETURN¶¬REMOVE_DEF_FROM_ENVIRONMENT (SOURCE_DEF);¶¬return;¶        end if;¶      end;¶    end if;¶¶¬      -- TYPE NAME IS NOT UN
IQUE¶¬      -- USE CHECK UNIQUE SUBPROGRAM TO¬GIVE ERROR MESSAGE¶    CHECK_UNIQUE_DEF (SOURCE_DEF);¶  end CHECK_TYPE_DEF;¶      --||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function ARE_HOMOGRAPH_HEADERS (HEADER_1, HEADER_2 : TREE) return
 BOOLEAN is¶¬      -- DETERMINES¬IF TWO HEADERS ARE HOMOGRAPHS¶¬      -- ONLY CALLED WITH HEADER FROM XD_HEADER ATTRIBUTE OF DEF¶¬      --¬ (HENCE DO
NOT NEED¬TO CHECK,¬E.G., DISCRETE_RANGE IN ENTRY)¶¶    KIND_1 : constant NODE_NAME := HEADER_1.TY;¶    KIND_2 : constant NODE_NAME := HEADER_2.TY;¶  b
egin¶¬      -- IF EITHER HEADER IS NON_OVERLOADABLE¶    if KIND_1 not in CLASS_SUBP_ENTRY_HEADER or KIND_2 not in CLASS_SUBP_ENTRY_HEADER then¶¶¬¬
-- THEY ARE HOMOGRAPHS¶      return True;¶¶¬¬      -- ELSE -- SINCE BOTH ARE¬OVERLOADABLE¶    else¶¶¬¬      -- SPLIT UP HEADER_2 AND CALL HEADER_IS_HOMOGR
APH¶      if KIND_2 = DN_FUNCTION_SPEC then¶        return HEADER_IS_HOMOGRAPH (HEADER_1, D (AS_PARAM_S, HEADER_2), D (AS_NAME, HEADER_2));¶      else
¶        return HEADER_IS_HOMOGRAPH (HEADER_1, D (AS_PARAM_S, HEADER_2));¶      end¬if;¶    end if;¶  end ARE_HOMOGRAPH_HEADERS;¶¶      --||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_SAME_PARAMETER_PROFILE (PARAM_S_1, PARAM_S_2
: TREE) return BOOLEAN is¶    PARAM_LIST_1¬     : SEQ_TYPE := LIST (PARAM_S_1);¶    PARAM_LIST_2¬     : SEQ_TYPE := LIST (PARAM_S_2);¶    PARAM_1, PAR
AM_2     : TREE;¶    ID_LIST_1, ID_LIST_2 : SEQ_TYPE := (TREE_NIL,¬TREE_NIL);¶    ID_1,¬ID_2¬     : TREE;¶  begin¶¬      -- LOOP THROUGH BOTH PARAMETE
R LISTS¶    loop¶¶¬¬      -- GET NEXT ELEMENT FROM PARAM_LIST_1, IF ANY¶      if IS_EMPTY (ID_LIST_1)¬then¶        if IS_EMPTY¬(PARAM_LIST_1) then¶¶¬¬¬¬
-- THERE IS NONE¶¬¬¬¬¬-- COMPATIBLE IF NO¬NEXT ELEMENT IN PARAM_LIST_2¶¬return IS_EMPTY (ID_LIST_2) and then IS_EMPTY (PARAM_LIST_2);¶        else¶¬POP
 (PARAM_LIST_1, PARAM_1);¶¬ID_LIST_1¬:= LIST (D (AS_SOURCE_NAME_S,¬PARAM_1));¶        end if;¶      end¬if;¶      POP¬(ID_LIST_1, ID_1);¶¶¬¬      -- GET
 NEXT ELEMENT FROM PARAM_LIST_2, IF ANY¶      if IS_EMPTY (ID_LIST_2)¬then¶        if IS_EMPTY¬(PARAM_LIST_2) then¶¶¬¬¬¬¬-- THERE IS NONE¶¬¬¬¬¬-- NOT CO
MPATIBLE SINCE THERE¬WAS AN ELEMENT ON PARAM_LIST_1¶¬return FALSE;¶        else¶¬POP (PARAM_LIST_2, PARAM_2);¶¬ID_LIST_2¬:= LIST (D (AS_SOURCE_NAME_S,
¬PARAM_2));¶        end if;¶      end¬if;¶      POP¬(ID_LIST_2, ID_2);¶¶¬¬      -- IF¬THEY ARE NOT OF THE¬SAME TYPE,¶      if GET_BASE_TYPE (D (SM_OBJ_T
YPE,¬ID_1)) /=¬GET_BASE_TYPE (D (SM_OBJ_TYPE, ID_2)) then¶¬¬¬  -- THEN¬THEY ARE NOT COMPATIBLE¶        return FALSE;¶      end¬if;¶    end loop;¶  end
 IS_SAME_PARAMETER_PROFILE;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  procedure
 CONFORM_PARAMETER_LISTS (PARAM_S_1, PARAM_S_2 :¬TREE) is¶    PARAM_LIST_1¬     : SEQ_TYPE := LIST (PARAM_S_1);¶    PARAM_LIST_2¬     : SEQ_TYPE := LI
ST (PARAM_S_2);¶    PARAM_1, PARAM_2     : TREE;¶    ID_LIST_1, ID_LIST_2 : SEQ_TYPE := (TREE_NIL,¬TREE_NIL);¶    ID_1,¬ID_2¬     : TREE;¶¶  begin¶¬
    -- IF PARAMETER LISTS ARE THE SAME¶    if PARAM_S_1 = PARAM_S_2 then¶¶¬¬      -- MUST BE FROM¬A GENERATED LIBRARY¬UNIT¶¬¬      -- ... SO, DO NOT CONFO

RM¬(I.E. DO NOT REMOVE¬DEFS)¶      return;¶    end if;¶¶¬      -- LOOP THROUGH BOTH PARAMETER LISTS¶    loop¶¶¬¬      -- CHECK THAT STRUCTURE OF LISTS I
S¬COMPATIBLE¶      if (IS_EMPTY (ID_LIST_1) xor IS_EMPTY (ID_LIST_2)) or¬(IS_EMPTY¬(PARAM_LIST_1) xor IS_EMPTY (PARAM_LIST_2)) then¶        exit;¶¶¬¬¬
  -- GET NEXT ELEMENT FROM PARAM_LISTS,¬IF ANY¶¬¬¬  -- RETURN IF NO MORE ELEMENTS¶      end¬if;¶      if IS_EMPTY (ID_LIST_1)¬then¶        if IS_EMPTY
¬(PARAM_LIST_1) then¶¬return;¶        else¶¬POP (PARAM_LIST_1, PARAM_1);¶¬POP (PARAM_LIST_2, PARAM_2);¶¬ID_LIST_1¬:= LIST (D (AS_SOURCE_NAME_S,¬PARAM_
1));¶¬ID_LIST_2¬:= LIST (D (AS_SOURCE_NAME_S,¬PARAM_2));¶¶¬if PARAM_1.TY /= PARAM_2.TY then¶¬  exit;¶¬end if;¶¶¬if not IS_COMPATIBLE_EXPRESSION (D (AS
_NAME, PARAM_1), D (AS_NAME, PARAM_2)) or¬else not IS_COMPATIBLE_EXPRESSION (D (AS_EXP, PARAM_1), D (AS_EXP, PARAM_2)) then¶¬  exit;¶¬end if;¶
end if;¶      end¬if;¶¶      POP¬(ID_LIST_1, ID_1);¶¶      if D (LX_SYMREP, ID_1) /= D (LX_SYMREP, HEAD (ID_LIST_2)) then¶        exit;¶      end¬if;¶
¶      POP¬(ID_LIST_2, ID_2);¶¶¬¬      -- ID'S ARE COMPATIBLE, REPLACE DEFS¶      D (SM_FIRST, ID_2, D (SM_FIRST, ID_1));¶      D (XD_REGION,¬ID_2, D (X
D_REGION,¬ID_1));¶      D (SM_INIT_EXP, ID_2, D¬(SM_INIT_EXP, ID_1));¶      D (SM_OBJ_TYPE, ID_2, D¬(SM_OBJ_TYPE, ID_1));¶      REMOVE_DEF_FROM_ENVIRO
NMENT (GET_DEF_FOR_ID¬(ID_2));¶¶    end loop;¶¶¬      -- INCOMPATIBLE, SINCE WE EXITED FROM LOOP¶    ERROR¬(D (LX_SRCPOS, PARAM_S_2), "PARAM LISTS NOT
 COMPATIBLE");¶¶¬      -- DISCARD DEFS FROM SECOND LIST ANYWAY¶    loop¶      while not IS_EMPTY (ID_LIST_2) loop¶        POP (ID_LIST_2, ID_2);¶
   REMOVE_DEF_FROM_ENVIRONMENT (GET_DEF_FOR_ID (ID_2));¶      end¬loop;¶      exit when IS_EMPTY (PARAM_LIST_2);¶      POP¬(PARAM_LIST_2, PARAM_2);¶
    ID_LIST_2 := LIST (D (AS_SOURCE_NAME_S, PARAM_2));¶    end loop;¶¶  end CONFORM_PARAMETER_LISTS;¶      --||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_COMPATIBLE_EXPRESSION (EXP_1, EXP_2¬: TREE) return BOOLEAN is¶¬      -- AR
GUMENTS ARE EXPRESSIONS OR RANGES OR VOID¶¬      -- RETURN TRUE IF COMPATIBLE (WITHIN PARAM OR DSCRMT LIST)¶  begin¶¬      -- $$$$$$$$ STUB $$$$$$$¶
  return True;¶  end IS_COMPATIBLE_EXPRESSION;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶
    --|¶  procedure MAKE_DEF_VISIBLE (DEF : TREE; HEADER : TREE := TREE_VOID)¬is¶  begin¶    D (XD_HEADER, DEF, HEADER);¶  end MAKE_DEF_VISIBLE;¶
 --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  procedure MAKE_DEF_IN_ERROR¬(DEF : TREE) i
s¶  begin¶    D (XD_HEADER, DEF, TREE_FALSE);¶  end MAKE_DEF_IN_ERROR;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||¶      --|¶  procedure REMOVE_DEF_FROM_ENVIRONMENT¬(DEF : TREE) is¶  begin¶    D (XD_HEADER, DEF, TREE_VOID);¶    D (XD_REGION_DE
F, DEF, TREE_VOID);¶    DI (XD_LEX_LEVEL, DEF, 0);¶    DB (XD_IS_USED,¬DEF, FALSE);¶  end REMOVE_DEF_FROM_ENVIRONMENT;¶      --||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function GET_DEF_EXP_TYPE (DEF : TREE) return TREE is¶    HEADER
```

```
  : constant TREE := D (XD_HEADER, DEF);¶    SOURCE_NAME : TREE¬  := D (XD_SOURCE_NAME, DEF);¶  begin¶    if HEADER.TY = DN_FUNCTION_SPEC then¶      r
eturn GET_BASE_TYPE (D¬(AS_NAME,¬HEADER));¶    elsif¬SOURCE_NAME.TY in CLASS_OBJECT_NAME then¶      return GET_BASE_TYPE (D¬(SM_OBJ_TYPE, D (XD_SOURCE
_NAME, DEF)));¶    elsif¬SOURCE_NAME.TY in CLASS_TYPE_SPEC then¶      if GET_BASE_TYPE (SOURCE_NAME).TY¬/= DN_TASK_SPEC then¶        Put_Line ("!! NON
 TASK TYPE NAME IN CALL TO GET_DEF_EXP_TYPE");¶        raise Program_Error;¶      end¬if;¶      return GET_BASE_TYPE (SOURCE_NAME);¶    else¶      ret
urn TREE_VOID;¶    end if;¶  end GET_DEF_EXP_TYPE;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|¶      --|¶  function GET_BASE_TYPE (TYPE_SPEC_OR_EXP_OR_ID : TREE) return TREE is¶    TYPE_SPEC : TREE := TYPE_SPEC_OR_EXP_OR_ID;¶  begin¶¶¬      --
 GET A TYPE¬SPEC FOR THE EXPRESSION OR ID¶    case TYPE_SPEC_OR_EXP_OR_ID.TY is¶      when DN_VOID =>¶        null;¶      when DN_USED_NAME_ID =>¶
    TYPE_SPEC := D (SM_DEFN, TYPE_SPEC);¶        if TYPE_SPEC /= TREE_VOID then¶¬TYPE_SPEC¬:= D (SM_TYPE_SPEC,¬TYPE_SPEC);¶        end if;¶      when
CLASS_OBJECT_NAME =>¶        TYPE_SPEC := D (SM_OBJ_TYPE, TYPE_SPEC);¶      when DN_FUNCTION_ID =>¶¬¬¬  -- (FOR¬SLICE WHOSE PREFIX IS FUNCTION WITH AL
L¬DEFAULT ARGS)¶        TYPE_SPEC := GET_BASE_TYPE (D (AS_NAME, D¬(SM_SPEC,¬TYPE_SPEC)));¶      when DN_PROCEDURE_ID =>¶¬¬¬  -- (FOR¬IDENTIFIER AS EXP
RESSION BEFORE OVERLOAD RESOLUTION)¶        TYPE_SPEC := TREE_VOID;¶      when DN_GENERIC_ID =>¶¬¬¬  -- (FOR¬EITHER OF¬THE ABOVE¬CASES)¶        if D (
XD_HEADER, GET_DEF_FOR_ID¬(TYPE_SPEC)).TY = DN_FUNCTION_SPEC then¶¬TYPE_SPEC¬:= GET_BASE_TYPE (D¬(AS_NAME,¬D (SM_SPEC, TYPE_SPEC)));¶        else¶¬TYP
E_SPEC¬:= TREE_VOID;¶        end if;¶      when CLASS_TYPE_NAME | CLASS_RANGE =>¶        TYPE_SPEC := D (SM_TYPE_SPEC, TYPE_SPEC);¶      when CLASS_US
ED_OBJECT | CLASS_EXP_EXP | DN_ATTRIBUTE¬| DN_FUNCTION_CALL | DN_INDEXED | DN_SLICE | DN_ALL =>¶        TYPE_SPEC := D (SM_EXP_TYPE, TYPE_SPEC);¶
 when DN_SELECTED =>¶        TYPE_SPEC := GET_BASE_TYPE (D (AS_DESIGNATOR, TYPE_SPEC));¶      when CLASS_TYPE_SPEC =>¶        null;¶      when DN_DISC
RETE_SUBTYPE =>¶        TYPE_SPEC := GET_BASE_TYPE (D (AS_NAME, D¬(AS_SUBTYPE_INDICATION, TYPE_SPEC)));¶      when DN_SUBTYPE_INDICATION =>¶        TY
PE_SPEC := D (SM_TYPE_SPEC, D¬(AS_NAME,¬TYPE_SPEC));¶      when CLASS_UNSPECIFIED_TYPE =>¶        null;¶      when others =>¶        Put_Line ("!! BAD
 PARAMETER FOR GET_BASE_TYPE");¶        raise Program_Error;¶    end case;¶¶¬      -- GET UNCONSTRAINED FOR CONSTRAINED TYPE¶¬      -- (IN CASE CONSTR
AINED¬PRIVATE WITH FULL TYPE VISIBLE)¶    if TYPE_SPEC.TY¬in CLASS_CONSTRAINED then¶      TYPE_SPEC := D (SM_BASE_TYPE, TYPE_SPEC);¶    end if;¶¶¬
  -- GET FULL TYPE SPEC FOR PRIVATE¬OR INCOMPLETE¶    if TYPE_SPEC.TY¬in CLASS_PRIVATE_SPEC then¶      if D (SM_TYPE_SPEC, TYPE_SPEC) /=¬TREE_VOID¬the
n¶        TYPE_SPEC := D (SM_TYPE_SPEC, TYPE_SPEC);¶      end¬if;¶    elsif¬TYPE_SPEC.TY = DN_INCOMPLETE then¶      if D (XD_FULL_TYPE_SPEC, TYPE_SPEC
) /= TREE_VOID then¶        TYPE_SPEC := D (XD_FULL_TYPE_SPEC, TYPE_SPEC);¶      end¬if;¶    end if;¶¶¬      -- LOOP TO GET BASE TYPE¶¬      -- $$$$ O
K? NON-TASK --> PRIVATE ?¶    while¬TYPE_SPEC.TY in CLASS_NON_TASK and then¬D (SM_BASE_TYPE, TYPE_SPEC) /= TYPE_SPEC loop¶      TYPE_SPEC := D (SM_BAS
E_TYPE, TYPE_SPEC);¶    end loop;¶¶    return TYPE_SPEC;¶  end GET_BASE_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||¶      --|¶  function GET_BASE_PACKAGE (PACKAGE_ID¬: TREE) return TREE¬is¶    UNIT_DESC : TREE := D (SM_UNIT_DESC, PACKAGE
_ID);¶    BASE_ID   : TREE;¶  begin¶    if UNIT_DESC.TY¬= DN_RENAMES_UNIT then¶      BASE_ID := GET_NAME_DEFN (D (AS_NAME, UNIT_DESC));¶      if BASE_
ID /=¬TREE_VOID¬then¶        return GET_BASE_PACKAGE (BASE_ID);¶      end¬if;¶    end if;¶    return PACKAGE_ID;¶  end GET_BASE_PACKAGE;¶¶   --|------
---------------------------------------------------------------------------------------¶end DEF_UTIL;¶
```